SCRIPT-CONTROLLED ACL – RESTRICT RECORD ACCESS BASED ON FIELD VALUE

PHASE 4: PROJECT PLANNING PHASE

The Project Planning Phase focused on preparing a systematic approach for implementing the ACL in ServiceNow. The required configuration steps, access conditions, script requirements, and testing activities were planned carefully. This planning helped ensure that the ACL could be developed and tested successfully according to the project requirements.

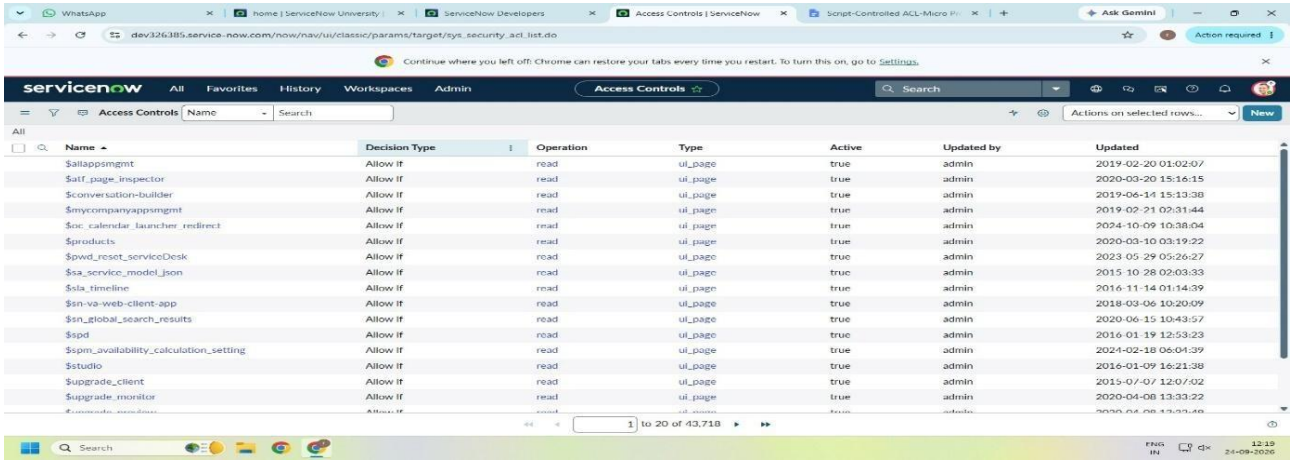
The planning phase prepared the work order before implementation. Each ACL setting, the operations to be covered, the script and the test cases were listed, so that the development phase could be carried out step by step without confusion.

4.1 Planning ACL Configuration

The required ACL configuration steps were planned based on the identified requirements. The table, access operation, conditions, and script requirements were considered before implementation.

The plan consisted of identifying the table (Institution Details), choosing the operations to be protected, deciding the required role, adding the data condition on the Branch field and adding the script. It was also decided to review the existing ACLs before creating new ones.

The screenshot shows the Access Controls list of the instance, which contains a very large number of system rules (over forty thousand). It shows the columns Name, Decision Type, Operation, Type, Active, Updated by and Updated, and it served as a reference for how standard ACLs are defined and named.

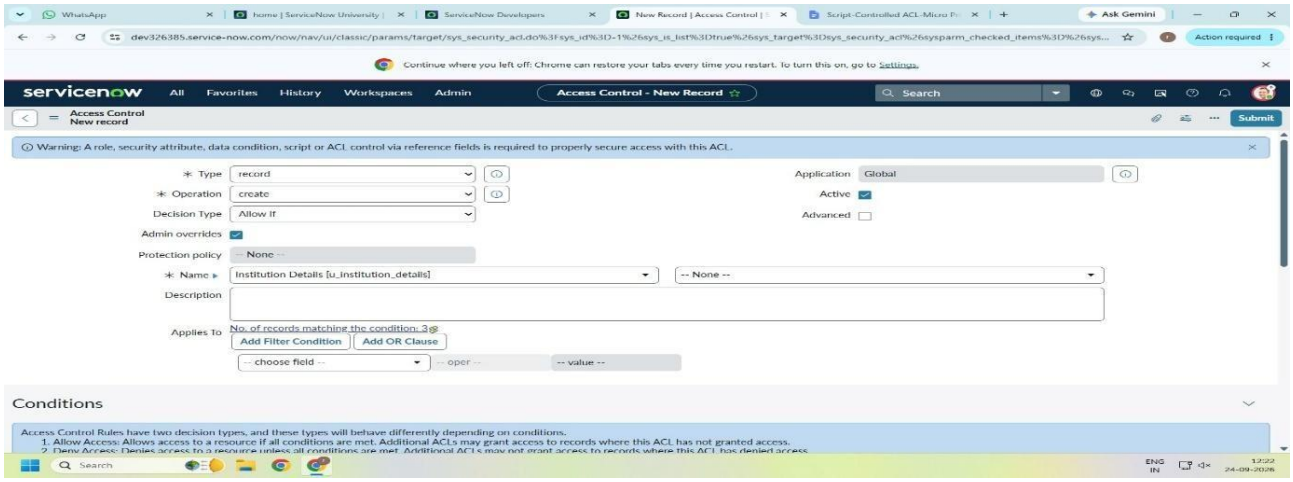


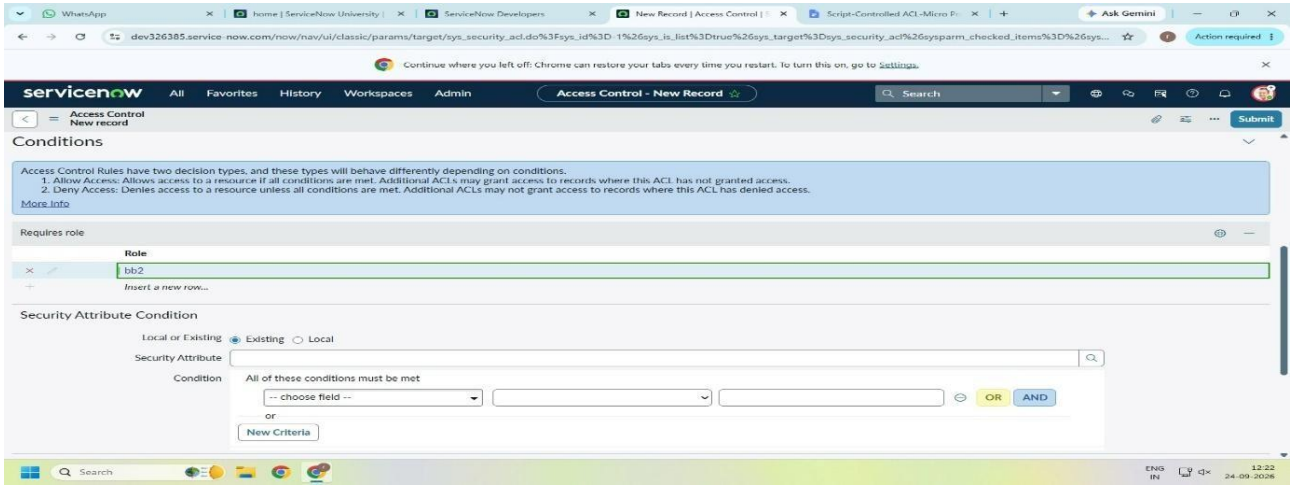
4.2 Planning Access Rules

The access rules were planned to determine how users would be allowed or restricted from accessing records.

An ACL protects one operation at a time, so separate rules were planned for the operations that matter: **read** (who can see the records), **create** (who can add new records), **write** (who can edit them) and **delete** (who can remove them). Each of these uses the same idea, an allow rule tied to the branch role and the script.

The first screenshot shows the ACL form with the operation set to *create* for the Institution Details table. The second one shows the **Requires role** section, where the custom role is added to the list, together with the Security Attribute Condition section. A user must hold this role for the rule to grant access.





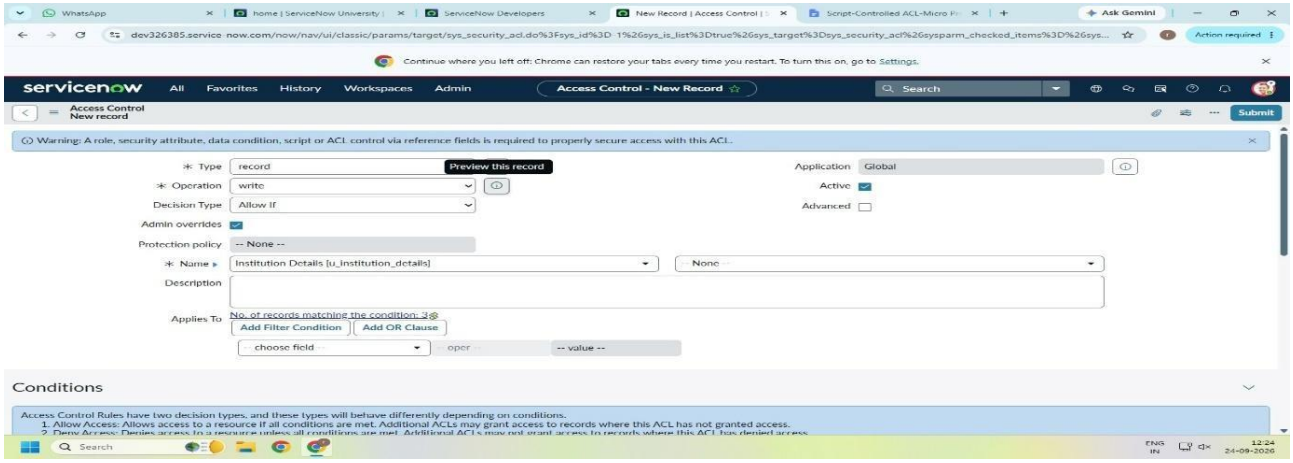
4.3 Planning the Script Logic

The script logic was planned to control record access dynamically based on the required conditions.

The script is written as a self-executing function that returns true or false. It uses the built-in *gs.hasRole()* method of the GlideSystem object to test the roles of the current user. Comments were added to each block so that anyone who opens the ACL later understands the reason for each decision. The planned script is shown below.

The screenshot shows the same ACL form with the operation set to *write*, which shows that the same logic was planned for editing the records.

```
(function () {
    // Allow admin users full access
    if (gs.hasRole('admin')) {
        return true;
    }
    // Allow only EEE branch users to see EEE records
    if (gs.hasRole('bb1')) {
        return true;
    }
    // Deny access for all others
    return false;
})();
```



4.4 Planning the Testing Process

The testing approach was planned to verify whether the configured ACL would provide the expected access and restrictions.

Testing was planned around three users: an administrator, a user with the branch role, and a user without the role. For each of them the list of the Institution Details table would be opened and the visible records would be compared with the expected result. Testing the negative case (a user who should not see anything) is as important as testing the positive one.